

PROJECT REPORT

SATELLITE RE-ENTRY HEAT ESTIMATOR

Student Name	Livin Leen
Register / Roll No.	26BAS10067
Program	B.Tech Aerospace Engineering
Semester	Ist Semester
Course / Project	Python Programming Project

Table of Contents

1. Introduction
2. Problem Statement
3. Functional Requirements
4. Non-functional Requirements
5. System Architecture
6. Design Diagrams
7. Design Decisions & Rationale
8. Implementation Details
9. Screenshots / Results
10. Testing Approach
11. Challenges Faced
12. Scope, Target Users & High-Level Features
13. Conclusion
14. Source Code / Project Files

1. Introduction

Satellite re-entry is a stage of a space mission in which a satellite or spacecraft returns through the Earth's atmosphere. During this process, the vehicle moves at high speed through air, producing aerodynamic effects and significant heating. Understanding these effects is useful for introducing aerospace engineering students to the relationship between velocity, atmospheric conditions, drag, kinetic energy and heat transfer.

The Satellite Re-entry Heat Estimator is a beginner-level Python program developed to provide a simplified numerical estimate of atmospheric conditions, drag force, kinetic energy and heat flux during re-entry. It also combines selected calculated values into a simple score and classifies the result as LOW, MODERATE or HIGH risk.

The project is intentionally designed as an educational estimator rather than a flight-certified or engineering-grade re-entry simulator. The current program uses a simplified atmospheric model and restricts the working altitude to 11,000 m.

2. Problem Statement

Aerospace engineering students often need to connect programming concepts with physical engineering quantities. For satellite re-entry, manually calculating several quantities can be repetitive and makes it harder for a beginner to see how the variables are connected. The project addresses this by creating one Python program in which the user enters re-entry and vehicle parameters and receives calculated atmospheric and aerodynamic values together with a simple risk classification.

3. Functional Requirements

- Accept re-entry velocity in m/s.
- Accept satellite mass in kg.
- Accept satellite surface area in m².
- Accept dynamic pressure in Pa as an input.
- Accept satellite altitude in m.
- Accept drag coefficient.
- Reject negative re-entry velocity, mass, surface area and altitude values.
- Limit the simplified model to 11,000 m when the entered altitude is above that value.
- Calculate atmospheric temperature and pressure.
- Calculate atmospheric density using the expression implemented in the Python source code.

- Calculate drag force and kinetic energy.
- Calculate heat flux using the simplified heat-flux expression in the source code.
- Calculate a risk score using heat flux, atmospheric density, velocity and drag force thresholds.
- Display the calculated values, total score and final risk level.

4. Non-functional Requirements

- **Usability:** The program should be simple enough for a beginner to run from a Python environment.
- **Readability:** Variable names and output labels should clearly indicate the physical quantity being handled.
- **Maintainability:** The program should keep input, validation, calculation, estimation and output stages logically separated.
- **Performance:** The calculations are lightweight and should execute almost immediately on a normal computer.
- **Reliability:** Basic invalid-input conditions should be detected before calculations are performed.
- **Portability:** The project should run in a standard Python 3 environment without requiring external packages; the source uses Python's built-in math module.

5. System Architecture

The system follows a simple sequential architecture. The user first supplies the input parameters. The program validates selected inputs, calculates atmospheric values, calculates aerodynamic and energy quantities, calculates heat flux, assigns a score based on threshold conditions and finally prints the results.

Class / Component Diagram

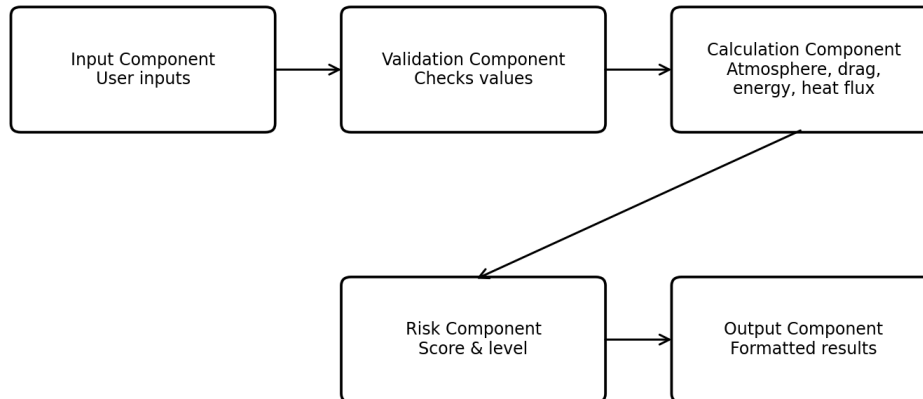


Figure 1. Component-level view of the Satellite Re-entry Heat Estimator.

5.1 Scope of the Project

The project focuses on a command-line Python estimator for educational use. It covers input collection, basic validation, simplified atmospheric calculations, drag and kinetic-energy calculations, heat-flux estimation and threshold-based risk classification. It does not attempt to model a complete re-entry trajectory, vehicle attitude, detailed atmospheric chemistry, ablation, or thermal protection system performance.

5.2 Target Users

- First-year aerospace engineering students learning Python.
- Students who want a simple application of programming to aerospace concepts.
- Beginners who want to understand how engineering inputs can be converted into calculated outputs.
- Students demonstrating a small Python project involving conditional statements and mathematical calculations.

5.3 High-level Features

- Interactive command-line input.
- Basic validation and boundary handling.
- Atmospheric temperature and pressure estimation.
- Atmospheric density calculation.
- Drag-force and kinetic-energy calculation.
- Heat-flux estimation.
- Multi-factor score calculation.
- LOW / MODERATE / HIGH risk output.

5.4 Source Code / Project Files

The main project file is a Python source file containing the complete implementation. The program imports the built-in math module, takes user inputs, performs validation and calculations, estimates risk and prints the final results. The project can be organized in a GitHub repository with the Python source file, README and this project report.

6. Design Diagrams

The following diagrams describe how the user interacts with the program and how the internal processing flows.

6.1 Use Case Diagram

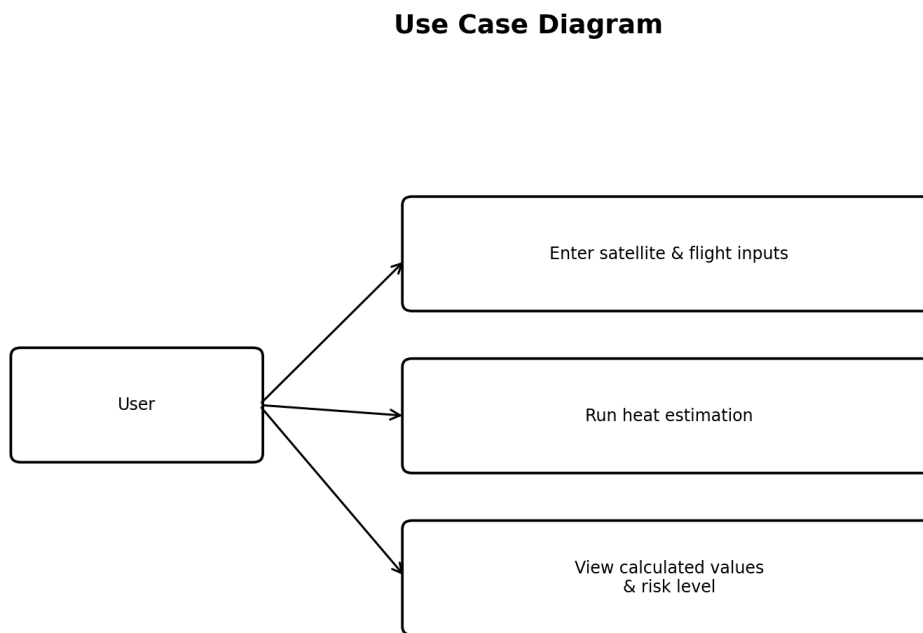


Figure 2. Use case diagram.

6.2 Workflow Diagram

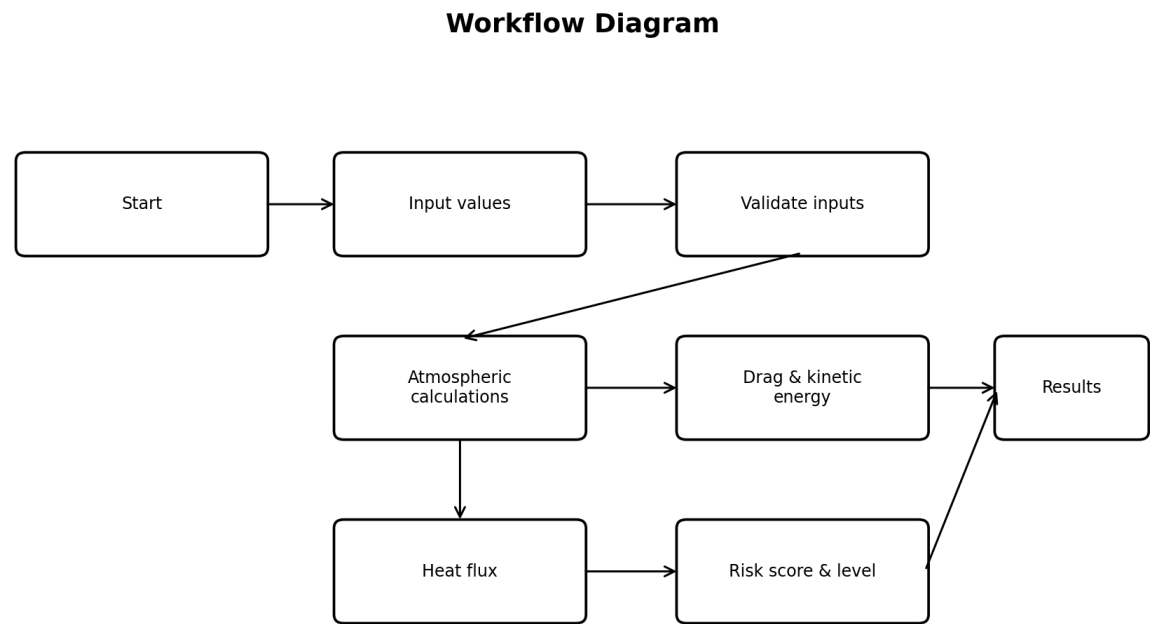


Figure 3. Overall workflow.

6.3 Sequence Diagram

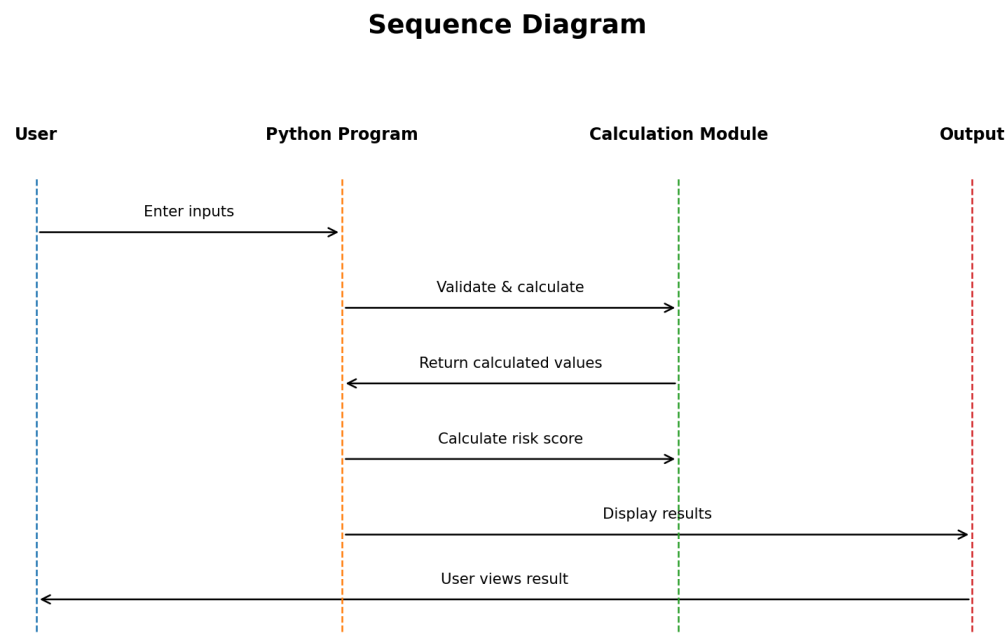


Figure 4. Sequence of interaction between the user, program, calculations and output.

6.4 Class / Component Diagram

Class / Component Diagram

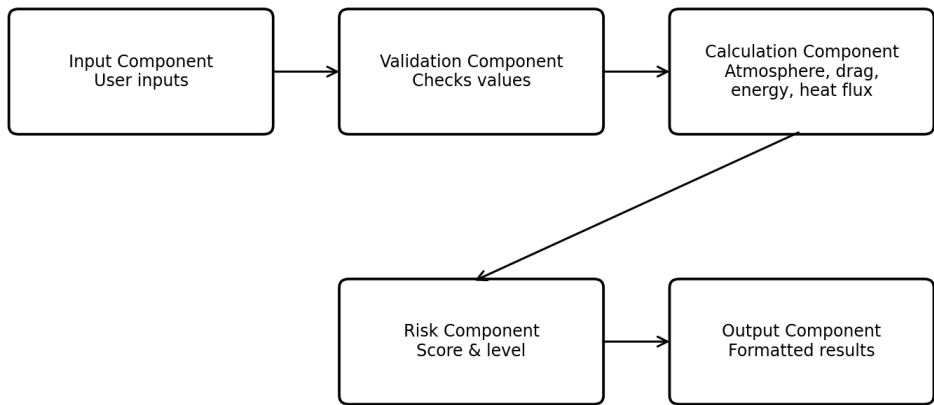


Figure 5. Component diagram.

6.5 ER Diagram

An ER diagram is not required for the current version because the program does not use a database or persistent data storage. All values are entered during execution and the calculated results are printed directly to the console.

7. Design Decisions & Rationale

Decision	Rationale
Command-line interface	A console interface keeps the project simple and allows the focus to remain on Python programming and aerospace calculations.
Simplified atmosphere	A basic atmospheric relationship is used so that a beginner can understand how altitude affects temperature and pressure.
Input validation	Negative values for quantities such as velocity, mass, area and altitude are rejected because they are not meaningful for the intended inputs.
11,000 m limit	The source code explicitly states that the simplified model is inaccurate above 11,000 m, so altitude is capped at that value.
Score-based risk	Instead of producing only one heat-flux value, the program considers several calculated indicators and converts them into a simple score.
No database	The project is a calculation utility and does not need stored user accounts, records or persistent data.

8. Implementation Details

The implementation is written in Python and uses the built-in math module. The program is divided conceptually into input, conditions, constant values, calculations, estimation and result-display stages.

8.1 Inputs

- Re-entry velocity (m/s)
- Satellite mass (kg)
- Surface area (m²)
- Dynamic pressure (Pa)
- Altitude (m)
- Drag coefficient

8.2 Validation and Boundary Conditions

The source checks whether re-entry velocity, mass, surface area and altitude are negative. If an invalid value is detected, an error message is printed and the program exits. If altitude is greater than 11,000 m, the program prints a warning and automatically sets altitude to 11,000 m.

8.3 Main Calculations

- Atmospheric temperature is calculated from the sea-level temperature, lapse rate and altitude.
- Atmospheric pressure is calculated using the pressure relationship implemented in the source code.
- Atmospheric density is calculated from the pressure, specific gas constant and atmospheric temperature using the expression currently present in the program.
- Drag force is calculated from density, drag coefficient, surface area and velocity.
- Kinetic energy is calculated from satellite mass and velocity.
- Heat flux is estimated using the simplified constant, atmospheric density and the cube of re-entry velocity.

8.4 Risk Estimation Logic

Parameter	Low	Moderate	High
Heat flux	≤ 6000	6000–12000	> 12000
Atmospheric density	< 0.1	0.1–<0.5	≥ 0.5
Re-entry velocity	≤ 5000	5000–8000	> 8000
Drag force	≤ 100000	100000–1000000	> 1000000

The program starts the score at zero and adds points according to four conditions:For each parameter, the program adds 1, 2 or 3 points. The final risk level is LOW when the score is 5 or below, MODERATE when it is between 6 and 9, and HIGH when it is above 9, following the conditions in the source code.

9. Screenshots / Results

The program prints a results section containing atmospheric temperature, atmospheric pressure, atmospheric density, drag force, kinetic energy, heat flux, estimated score and risk level. A final submission can include a screenshot of the terminal/IDLE window showing a successful run.

Result : Low Risk

Parameters	Values
Re-entry Velocity	100 m/s
Weight of satellite	100 Kg
Surface area	0.001 m ²
Dynamic pressure	100 Pa
Satellite altitude	11000m
Drag coefficient	0.1
Atmospheric Temperature	216.65 K
Atmospheric pressure	22620.12 Pa
Atmospheric Density	0.364 Kg/m ³
Drag force	0.18N
Kinetic Energy	500000 J

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\livin\OneDrive\Desktop\Satellite Re-entry Heat Estimator> python
main.py
-----SATELLITE RE-ENTRY HEAT ESTIMATOR-----
Re_entry velocity of the satellite (m/s): 100
Weight of the satellite (Kg):100
Surface area of the satellite (m^2): 0.001
Dynamic pressure (Pa): 100
Satellite altitude (m): 11000
Drag coefficient: 0.1
-----RESULTS-----
Atmospheric Temperature: 216.65 K
Atmospheric Pressure: 22620.12 Pa
Atmospheric Density: 0.364 Kg/m^3
Drag Force: 0.18 N
Kinetic Energy: 500000.0 J
Estimated Score: 5
Risk Level: LOW
PS C:\Users\livin\OneDrive\Desktop\Satellite Re-entry Heat Estimator> |
```

Result: Moderate Risk

Parameters	Values
Re-entry Velocity	6000 m/s
Weight of satellite	1000 Kg
Surface area	10 m^2
Dynamic pressure	100 Pa
Satellite altitude	10000 m
Drag coefficient	1
Atmospheric Temperature	223.15 K
Atmospheric pressure	26423.76 Pa
Atmospheric Density	0.413 Kg/m^3
Drag force	74252763.71 N
Kinetic Energy	18000000000 J

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\livin\OneDrive\Desktop\Satellite Re-entry Heat Estimator> python main.py
-----SATELLITE RE-ENTRY HEAT ESTIMATOR-----
Re_entry velocity of the satellite (m/s): 6000
Weight of the satellite (Kg):1000
Surface area of the satellite (m^2): 10
Dynamic pressure (Pa): 100
Satellite altitude (m): 10000
Drag coefficient: 1
-----RESULTS-----
Atmospheric Temperature: 223.15 K
Atmospheric Pressure: 26423.76 Pa
Atmospheric Density: 0.413 Kg/m^3
Drag Force: 74252763.71 N
Kinetic Energy: 18000000000.0 J
Estimated Score: 9
Risk Level: MODERATE
PS C:\Users\livin\OneDrive\Desktop\Satellite Re-entry Heat Estimator> |
```

Result: High Risk

Tt Topic	Tt Decision
Re-entry Velocity	9000 m/s
Weight of satellite	1000 Kg
Surface area	425 m^2
Dynamic pressure	100 Pa
Satellite altitude	10000 m
Drag coefficient	1
Atmospheric Temperature	223.15 K
Atmospheric pressure	26423.76 Pa
Atmospheric Density	0.413 Kg/m^3
Drag force	7100420529.78 N
Kinetic Energy	40500000000 J

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\livin\OneDrive\Desktop\Satellite Re-entry Heat Estimator> python main.py
-----SATELLITE RE-ENTRY HEAT ESTIMATOR-----
Re_entry velocity of the satellite (m/s): 9000
Weight of the satellite (Kg):1000
Surface area of the satellite (m^2): 425
Dynamic pressure (Pa): 100
Satellite altitude (m): 10000
Drag coefficient: 1
-----RESULTS-----
Atmospheric Temperature: 223.15 K
Atmospheric Pressure: 26423.76 Pa
Atmospheric Density: 0.413 Kg/m^3
Drag Force: 7100420529.78 N
Kinetic Energy: 40500000000.0 J
Estimated Score: 10
Risk Level: HIGH
PS C:\Users\livin\OneDrive\Desktop\Satellite Re-entry Heat Estimator> |
```

10. Testing Approach

Testing was planned around normal values, boundary values and invalid values.

Test Case	Input Condition	Expected Behaviour	Status
TC-01	Normal positive inputs	Program calculates and displays results	To be verified with final run
TC-02	Negative velocity	Error message and program exit	Implemented in source
TC-03	Negative mass	Error message and program exit	Implemented in source
TC-04	Negative surface area	Error message and program exit	Implemented in source
TC-05	Negative altitude	Error message and program exit	Implemented in source
TC-06	Altitude > 11,000 m	Warning shown and altitude set to 11,000 m	Implemented in source
TC-07	Different heat-flux ranges	Score changes according to thresholds	Implemented in source
TC-08	Different density ranges	Score changes according to thresholds	Implemented in source
TC-09	Different velocity ranges	Score changes according to thresholds	Implemented in source
TC-10	Different drag-force ranges	Score changes according to thresholds	Implemented in source

11. Challenges Faced

- Connecting programming concepts such as input(), if/elif/else and arithmetic expressions with aerospace quantities.
- Choosing simple threshold values for a beginner-friendly risk classification.
- Handling invalid input values without allowing the program to continue with clearly invalid values.
- Keeping the atmospheric model simple enough for a first-year project.
- Understanding that a heat estimate is different from a complete re-entry thermal analysis.
- Organizing several calculations into a readable sequence so that the final output is easy to interpret.

12. Scope, Target Users & High-Level Features

12.1 Scope

The current scope is a small educational command-line estimator. It demonstrates the use of Python for an aerospace engineering problem and provides a simple way to observe how changes in the input parameters can affect calculated quantities and the resulting score.

12.2 Target Users

The primary target users are first-year aerospace engineering students and beginners in Python programming who want an application-based project.

12.3 High-level Features

- Interactive input
- Validation of selected inputs
- Atmospheric estimation
- Aerodynamic calculations
- Heat-flux estimation
- Risk score generation
- Clear console output

13. Conclusion

The Satellite Re-entry Heat Estimator demonstrates how basic Python programming can be applied to an aerospace engineering problem. The project takes a set of satellite and re-entry parameters, performs a sequence of calculations and presents the results in a simple format. The addition of a threshold-based score gives the program an easy-to-understand way of classifying the estimated conditions.

The project is intentionally a simplified educational model. It should not be treated as a substitute for validated aerospace simulation software or a detailed thermal analysis. Its main purpose is to demonstrate programming logic, mathematical calculations, input validation and the application of basic aerospace concepts in a small Python project.

14. Source Code / Project Files

The following is the core structure represented by the submitted Python file:

```
import math

print("-----SATELLITE RE-ENTRY HEAT ESTIMATOR-----")

# inputs
Re_entry_Velocity = float(input("Re-entry velocity of the satellite(m/s):"))
Mass_of_the_satellite = float(input("weight of the satellite(Kg):"))
Surface_area = float(input("Surface area of the satellite(m^2):"))
Dynamic_pressure = float(input("Dynamic pressure(Pa):"))
Altitude = float(input("Satellite altitude(m):"))
Drag_coefficient = float(input("Drag coefficient:"))

# validation, atmospheric calculations,
# drag/energy/heat calculations,
# risk score and result display follow.
```

The complete Python source file should be included in the GitHub repository and submitted according to the project instructions.

Appendix A – Implementation Notes

The report describes the current source implementation rather than replacing it with a different model. In the submitted code, `Dynamic_pressure` is collected as an input but is not subsequently used in the calculations. Also, the atmospheric-density expression and the negative-density assignment use the exact variable/expression forms present in the source. These are useful points to review before the final project demonstration because they can affect numerical results.

The source also prints Drag Force twice in the results line. This is a presentation-level issue and can be cleaned up before final submission if desired.

Reference

- 1)VITyarthi - Build Your Own Project: General Project Instructions & Submission Guidelines, VIT Bhopal University.
- 2)Python 3 Documentation - Python programming language and math module reference.
- 3)Project Source Code - `main.py`, Simplified Satellite Re-entry Heat Estimator
- 4)Project Statement-`statement.md`.
- 5)Project README-`README.md`.